

Interconnection Structures

Condensed Study Notes

Generated: 2026-05-24 23:44 UTC

COMPUTER ARCHITECTURE & ORGANIZATION LECTURE (WEEK 2)

Welcome back! This week, we're diving deeper into how computers are built and how they work internally.

Imagine a computer as a highly organized office.

- The **CPU (Central Processing Unit)** is like the main manager or the 'brain' of the office. It's the part that does all the thinking, calculating, and decision-making. When you tell your computer to do something, the CPU is the one figuring out how to get it done.
- The **memory** is like the office's filing cabinets. It's where the computer stores all the information it needs to work with, like documents, programs, and data. There are different types of memory, and some are faster to access than others, just like keeping frequently used files on your desk versus in a deep archive.
- **Input/Output (I/O) devices** are like the ways people in the office communicate with the outside world or get things done. This includes things like your keyboard and mouse (input) and your screen or printer (output). They are the tools that allow the computer to receive instructions and show you the results.

These are the fundamental parts that make a computer run. The way these parts are designed and connected is what we call computer architecture and organization.

Interconnection Structures

You know how the computer has a few main parts: the processor (the brain), memory (where it keeps stuff), and I/O devices (how it talks to you and the world)? Well, these parts need to be able to talk to each other.

Imagine a busy office. The processor is like the manager, the memory is like the filing cabinets, and the I/O devices are like the phones and mail slots. For everyone to get their work done, there need to be paths for them to send messages and get things to each other – like hallways, phone lines, and a mail delivery system.

In a computer, the collection of these paths that connect the processor, memory, and I/O devices is called the **interconnection structure**. The way these paths are set up really matters because it affects how easily and quickly these parts can exchange information.

There are different ways to design these paths. One way is to have a **single bus**. Think of this like a single main road where all the traffic between the office departments has to go. It's simple, but it can get crowded and slow things down if too many things are happening at once.

Another way is to use **multiple buses**. This is like having several different roads or even dedicated phone lines between specific departments. This can make things much faster because different parts can communicate simultaneously without getting in each other's way. The specific design of the interconnection structure depends on what kind of communication needs to happen between the

computer's modules.

I/O module

Imagine your computer has different ways of talking to the outside world, like sending a letter or receiving a package. These ways are managed by something called an 'I/O module'.

From the inside of the computer, interacting with these external devices is a bit like interacting with memory. The I/O module is responsible for handling the communication between the computer's main parts (like the processor and memory) and the devices you see and use (like your keyboard or a printer).

Think of the I/O module as a special post office that handles mail for specific addresses (external devices). Each of these external devices has a unique 'port' or address, just like a house number, so the computer knows where to send or receive information.

Here's how the different parts of the computer might 'talk' to each other, with the I/O module playing a role:

- **Processor to Memory:** The processor (the computer's brain) can write information to the memory.
- **Memory to Processor:** The processor can read instructions or data from the memory.
- **I/O to Processor:** The processor can read data that comes from an external device, through the I/O module.
- **Processor to I/O:** The processor can send data to an external device, using the I/O module.

Sometimes, the I/O module needs to send a signal to the processor to get its attention, like a doorbell ringing. This is called an 'interrupt signal'.

There are two main types of actions an I/O module can perform: 'read' (getting information) and 'write' (sending information). An I/O module can also manage more than one external device.

In some cases, the I/O module can even send data directly to or from the memory, without the processor having to be involved in every step. This is called 'Direct Memory Access' (DMA). It's like sending a package directly between two storage rooms without asking the manager (processor) to carry it each time.

Memory

Imagine memory as a vast collection of mailboxes, each with a unique number. When the computer needs to store or retrieve something, it needs to know which mailbox to use.

- **Address:** This is like the unique number on each mailbox. It tells the computer exactly where a piece of information is located within the memory.
- **Operation:** The computer can either put something into a mailbox (write) or take something out of a mailbox (read). The type of action is determined by control signals, which are like instructions telling the memory whether to read or write.
- **Word:** This is a single piece of information that can be stored or retrieved from a mailbox. Think of it as a letter or a package that fits into one mailbox.

- **Memory Module:** Typically, a memory module is organized into many of these mailboxes, all designed to hold the same size 'word' of data. Each mailbox is assigned a specific, sequential number, starting from 0 up to a certain limit (like N-1, where N is the total number of mailboxes).

Complex Instruction Set Architecture (CISC)

We've seen how computers use instructions to perform tasks. Now, let's talk about different ways the 'brain' of the computer, the CPU (Central Processing Unit), can be designed to handle these instructions.

Imagine you have a very complicated chore, like baking a multi-layer cake. You could either have a very simple tool that does just one thing (like a whisk) and use it many times, or you could have a more complex, specialized machine that does many steps at once (like a high-tech baker's assistant that can mix, bake, and frost).

Complex Instruction Set Architecture (CISC) is like that specialized baker's assistant. The main idea here is to make the CPU's hardware more complex so that a single, powerful instruction can do a lot of work.

Think of a command like 'multiply'. In a CISC design, this single 'multiply' instruction might internally handle:

- **Loading:** Getting the numbers you want to multiply from memory (like picking ingredients from the pantry).
- **Evaluating:** Actually performing the multiplication.
- **Storing:** Putting the result back into memory (like putting the finished cake on a shelf).

So, instead of needing separate instructions for 'load this number', 'load that number', 'multiply them', and 'store the answer', one complex instruction does it all. This makes the 'language' the CPU understands (its instruction set) very rich and capable of handling intricate operations with fewer, but more powerful, commands.

Instruction set Architecture: CISC, RISC

We've already touched on how instructions are like commands that tell the computer's brain (the CPU) what to do. Now, let's think about how these command lists, or 'instruction sets', are designed.

Imagine you have a very complicated task, like building a piece of furniture. You could have a manual with one super-detailed instruction: "Assemble the entire bookshelf, including attaching all shelves and back panel." Or, you could have a manual with many simpler instructions: "Attach side panel A to base panel B," then "Insert shelf 1," then "Secure back panel," and so on.

This is similar to how different computer processors are designed. The way they handle these command lists is called their **Instruction Set Architecture (ISA)**. It's essentially the vocabulary and grammar that the CPU understands.

We've already met **Complex Instruction Set Architecture (CISC)**. Remember how in CISC, one instruction can do a lot of work, like loading data from memory, doing a calculation, and then saving the result, all in one go? This is like our first furniture example – one big, complex command.

Now, let's look at the other main approach: **Reduced Instruction Set Architecture (RISC)**. In RISC, the philosophy is to use simpler, more basic instructions. Each instruction does just one small thing, like "load data from memory" or "add two numbers" or "store data to memory." To achieve the same complex task as a single CISC instruction, a RISC processor would need a sequence of these simpler instructions.

Think of it like a chef. A CISC chef might have a special tool that can chop, dice, and sauté all at once. A RISC chef would have separate tools for chopping, dicing, and sautéing, and would use them one after another. While the RISC chef might use more individual steps, each step is very quick and straightforward, and the tools (instructions) themselves are simpler and easier to design and manage.

Reduced Instruction Set Architecture (RISC)

Imagine you have a toolbox. In one toolbox (CISC, which we talked about before), you have very specialized tools that can do a lot of things with a single action, like a fancy multi-tool. In another toolbox (RISC), you have simpler, more basic tools, where each tool does just one job very well.

The main idea behind RISC is to make the computer's 'brain' (the CPU) simpler by giving it a set of commands that are very basic and do just a few things at a time.

Instead of one complex command that might load data, do a calculation, and then save the result all at once, RISC breaks these down into individual, simple commands.

For example:

- A 'load' command will just fetch data from memory.
- A 'store' command will just save data to memory.
- Other commands will handle the actual calculations or evaluations.

By using these very basic steps, the hardware that runs these commands can be made simpler and often faster because it doesn't have to handle as many complex, combined actions.

The “best” way to design a CPU has been a subject of debate

The way computer brains (CPUs) are designed has been a topic of discussion for a long time. It's a bit like deciding whether to have a toolbox with a few very specialized, complex tools, or a larger toolbox with many simple, basic tools.

This choice is about how the CPU understands and carries out its commands, which we call its **instruction set**.

There are two main approaches to designing these instruction sets:

- **CISC (Complex Instruction Set Computer)**: Imagine a super-specialized tool that can do a whole bunch of things in one go. For example, one CISC command might be able to grab information from memory, do a calculation, and then put the result back, all with a single instruction. This makes the hardware more complex because it has to handle these big, multi-step actions.
- **RISC (Reduced Instruction Set Computer)**: Now imagine a set of very simple, basic tools. Each tool does just one thing very efficiently. With RISC, complex tasks are broken down into many small, simple

commands. This means the CPU might need more individual commands to complete a task, but each command is faster and simpler for the hardware to process.

The big question is: should commands be long and powerful (CISC), doing more in fewer steps but needing more complex hardware, or should they be short and simple (RISC), requiring more steps but potentially being more efficient overall?

Characteristic of RISC

We've talked about how RISC (Reduced Instruction Set Architecture) uses simpler instructions compared to CISC.

Here are some of the key traits that make RISC designs work well:

- 1. Simpler Instructions, Easier to Understand:** Imagine a very detailed instruction manual versus a simple checklist. RISC instructions are like the checklist – they are straightforward and easy for the CPU to figure out what to do with them. This makes the process of 'decoding' (understanding) the instruction much simpler.
- 2. Instructions Fit in One 'Word':** Think of a 'word' as a standard-sized box for holding information. RISC instructions are designed to fit neatly into one of these boxes. This makes it easier and faster for the CPU to fetch and process them.
- 3. Quick Execution:** Each simple RISC instruction is designed to be completed very quickly, ideally within a single 'clock cycle'. A clock cycle is like a tiny tick of a metronome for the computer; the faster the ticks, the faster things can happen. So, completing an instruction in one tick is super efficient.
- 4. Plenty of 'Scratchpads' (Registers):** Imagine a chef having lots of small plates right next to them to hold ingredients they're currently using. In a computer, these are called **registers**, which are small, super-fast storage locations inside the CPU. RISC designs tend to have more of these general-purpose registers, allowing the CPU to keep more temporary data handy without having to constantly go back to main memory (like a bigger pantry).
- 5. Simple Ways to Find Data (Addressing Modes):** 'Addressing modes' are like different methods for telling the CPU where to find the data it needs. RISC uses very simple and consistent ways to do this. This makes it easier for the CPU to figure out where the data is located quickly.
- 6. Fewer Types of Data:** RISC designs typically handle fewer basic types of data directly. This simplicity helps keep the instructions and the hardware that processes them straightforward.
- 7. Pipelining is Possible:** Think of an assembly line in a factory. Instead of building one car completely before starting the next, an assembly line has different stations working on different cars simultaneously. **Pipelining** is a technique where the CPU can start working on a new instruction before it finishes the previous one, by breaking down the instruction execution into stages. RISC's simple, uniform instructions make this assembly-line approach much easier and more efficient to implement.

Characteristic of CISC

We've talked about how computer commands (called **instructions**) can be simple or complex. Now let's dive into what makes CISC (Complex Instruction Set Architecture) instructions, well, complex!

Think of a recipe. A CISC instruction is like a super-detailed recipe that combines many small steps into one big, powerful instruction. For example, if you wanted to add two numbers that are stored in different places, a CISC approach might have a single instruction that says, "Go get the first number, go get the second number, add them together, and put the result back where you found them." This is all done with just one command!

Here's what that looks like in terms of characteristics:

- **Complex Instructions:** These instructions are so powerful they can do multiple things at once. This makes the process of figuring out what the instruction means (called **instruction decoding**) more complicated for the computer's hardware.
- **Larger Instructions:** Because a single CISC instruction can do so much, it often needs more space to be written down than a simple instruction. Imagine writing a whole paragraph versus just one word – the paragraph takes up more space. So, CISC instructions can be longer than what the computer can handle in one go (called **one-word size**).
- **Takes Longer to Run:** Since one instruction is doing so many things, it can take more time for the computer's clock (which keeps the beat of the computer, measured in **clock cycles**) to finish executing that single, complex instruction.
- **Fewer Storage Spots for Data:** In CISC, since instructions can directly work with data stored in the computer's main storage (called **memory**), the CPU doesn't need as many small, quick storage spots for temporary data (called **general-purpose registers**). The work happens more directly in memory.
- **More Ways to Find Data (Addressing Modes):** CISC offers many different ways for an instruction to specify where the data it needs is located. This is like having many different ways to describe an address – you could give a street name, a landmark, or even a GPS coordinate. This flexibility is called **complex addressing modes**.
- **Handles More Types of Data:** CISC designs often have built-in ways to handle a wider variety of data types (like different sizes of numbers or text).

CISC vs. RISC: Which is Better?

We've talked about two main ways to design the 'language' a computer's brain (the CPU) understands: CISC and RISC.

Think of it like giving instructions to someone. With CISC, you give one very detailed instruction that might say, "Go to the store, buy milk and eggs, and bring them back." It's powerful because one command does a lot.

With RISC, you break that down into simpler steps: "1. Go to the store. 2. Buy milk. 3. Buy eggs. 4. Come back." Each step is simpler, but you need more steps to get the whole job done.

Here's how this applies to computer design:

- **CISC (Complex Instruction Set Architecture):** This approach tries to get more done with fewer instructions. It's like having those super-detailed, multi-step instructions. The goal is to reduce the total number of instructions needed for a program.
- **RISC (Reduced Instruction Set Architecture):** This approach focuses on making each instruction very simple and fast to execute. It aims to reduce the number of 'cycles' (the basic beats of the computer's clock) needed for each instruction. This means you might need more instructions in total for a program, but each one finishes quicker.

So, which is better?

There's no single "correct" answer! Both have their strengths and weaknesses:

- **Historically:** CISC became popular when programming was done in 'assembly language' (a very low-level way of telling the computer what to do). CISC made assembly programming easier by letting one instruction do many things. But as 'high-level languages' (like Python or Java) became more common, the need for complex single instructions lessened, and RISC gained traction.
- **Real-world examples:** You'll find CISC in processors like the x86 (used in most home computers). RISC is found in processors like ARM (common in smartphones and tablets), PowerPC, and Sparc.

In short, CISC packs more into each instruction, while RISC makes each instruction simpler and faster, potentially requiring more instructions overall.

Difference

Now that we've looked at CISC and RISC, let's break down the key ways they differ. Think of it like two different approaches to building a toolbox.

CISC (Complex Instruction Set Architecture) is like a toolbox filled with very specialized, multi-purpose tools. You might have one tool that can both hammer a nail and saw wood. This means fewer tools are needed, but each tool is quite complex and might take a bit longer to figure out how to use for a specific job.

RISC (Reduced Instruction Set Architecture) is like a toolbox with many simple, single-purpose tools. You have a hammer for hammering, a saw for sawing, and a screwdriver for screwing. This means you need more tools (instructions) overall, but each tool is very straightforward and quick to use for its specific task.

Here's a quick rundown of their differences:

Focus: CISC focuses more on making the hardware (the physical parts) do more complex work with fewer instructions. RISC focuses more on making the software* (the programs) simpler by using many basic instructions.

- **Instructions:** CISC instructions can do multiple things at once, like grabbing data from memory, doing a calculation, and putting the result back. RISC instructions are simpler and usually focus on just one thing at a time, like just grabbing data, or just doing a calculation. This means RISC needs more of these simple instructions to do the same job as one complex CISC instruction.
- **Instruction Size:** CISC instructions can be of varying lengths, like having tools of different shapes and sizes. RISC instructions are typically all the same size, making them easier and faster for the CPU

to handle, like having all tools fit neatly into specific slots.

Execution Speed: *Because RISC instructions are simple, the CPU can often execute one instruction in a single clock cycle** (a tiny unit of time the computer uses to synchronize operations). CISC instructions, being more complex, often take more than one clock cycle to complete.

Registers: Registers* are small, super-fast storage areas inside the CPU itself. CISC designs generally need fewer registers because their complex instructions can handle data directly. RISC designs, on the other hand, need more registers to hold all the temporary data that their many simple instructions need to work with. This is why CISC uses transistors for storing complex operations, while RISC uses them for more registers.

Code Size: *Since CISC instructions are powerful and can do a lot, you might need fewer of them to write a program, resulting in a code size** (the total amount of program instructions) that is smaller. RISC, with its simpler, single-purpose instructions, often requires more instructions to achieve the same outcome, leading to a larger code size.

- **Operations:** CISC can perform operations involving data directly from memory or between memory locations. RISC is more restricted and typically only performs operations on data that is already in its registers (REG to REG operations). It can also do REG to MEM (register to memory) or MEM to MEM (memory to memory) operations, but these are less common than in CISC.

Instruction and Instruction Types

We've talked about how CPUs can have either lots of complex commands (CISC) or fewer, simpler commands (RISC). But even within those philosophies, there are ways to describe the commands themselves, which we call the 'instruction set'.

The instruction set is like the official dictionary of words that the CPU understands. It's the way the software (like your apps) talks to the hardware (the physical parts of the computer) to tell it what to do.

Instructions can be described in a few ways:

Addressing Modes: *This is like saying how** the CPU should find the information it needs to work with. Think of it like giving directions to find a specific item. Should it look in a fixed spot? Should it calculate where to find it? Or something else?

- **Numbers of Operands:** An 'operand' is simply the piece of data that an instruction acts upon. This characteristic describes how many pieces of data an instruction can work with at once. Some instructions might work on one piece of data, others on two, and so on. It's like saying if a command is 'add this number' (one operand) or 'add this number to that number' (two operands).

- **Types of Operations Supported:** This refers to the actual actions the instruction can perform. Can it add numbers? Can it move data around? Can it make decisions? This is the core function of the instruction.

- **Fixed versus Variable Length:** This describes how long each instruction is. Some CPUs have instructions that are always the same length (fixed), like a standard-sized Lego brick. Others can have instructions that vary in length (variable), like different sized pieces of playdough. This can affect how the CPU processes them.

Word length

Imagine a computer's processor, the 'brain' that does all the thinking, can be thought of in terms of how much information it can handle at once. This is often described by its 'word length'.

Think of 'word length' like the size of a carrying case a worker uses. A 4-bit worker might have a tiny case, an 8-bit worker a slightly bigger one, and so on, up to 32-bit or even larger cases. The size of this case determines how much information (data) they can carry or process in a single trip.

Sometimes, the size of the data a processor works with (the 'data word'), the commands it understands ('instruction'), and the locations it can find information ('address') are all the same length. However, especially in older or simpler designs, the instructions or addresses might need to be longer than the basic data size, like needing a longer piece of paper to write down instructions than the actual data you're recording.

Separately from the set of commands a CPU understands (its 'instruction set'), we can also talk about how it actually *executes* those commands. A 'single-issue processor' is like a worker who can only do one task at a time, even if they are juggling a few different steps of a task. A 'multiple-issue processor', like a 'superscalar processor', is more advanced. It has special logic that can figure out which tasks can be done at the exact same time, allowing it to work on several things simultaneously, making it much faster.

The 'programming model', also called the 'programmer model', is like the set of tools or containers that a programmer can directly use when writing instructions for the CPU. The CPU itself has many other internal 'registers' (small, super-fast storage locations) that it uses for its own work, but programmers can't directly access them.

INSTRUCTION SEQUENCING

Imagine a computer's processor is like a diligent student working through a list of tasks. This section is about how the processor moves from one task to the next.

How Instructions are Executed:

When a processor runs a program, it's essentially following a list of instructions. This process of running through these instructions is broken down into two main steps:

- 1. Instruction Fetch (Getting the Task):** The processor first needs to grab the next instruction from its memory. Think of the PC (Program Counter) as a bookmark in a book, always pointing to the next instruction to be read. The processor uses the address from the PC to find the instruction in memory and brings it into a special holding spot inside the processor called the Instruction Register (IR).
- 2. Instruction Execute (Doing the Task):** Once the instruction is in the IR, the processor figures out what it needs to do. It examines the instruction to understand the specific operation required.

Straight-Line Sequencing:

Normally, the processor just goes through the instructions one after another, in the order they appear in memory. It moves from one instruction to the next based on the address in the PC, incrementing the PC each time. This simple, step-by-step progression is called 'straight-line sequencing'. It's like reading a book

from beginning to end without skipping pages.

Types of Instructions:

Computers use different types of instructions to get things done. These instructions can be categorized in a few ways:

- **Basic Instruction Types:** These are the fundamental actions a processor can perform.
- **Data transfer between memory and processor registers:** Moving information back and forth between the main memory and the processor's internal storage areas (registers).
- **Arithmetic & logic operations on data:** Performing calculations (like addition) or making decisions based on data.
- **Program sequencing & control:** Instructions that change the normal flow of execution, like jumping to a different part of the program.
- **I/O transfers:** Moving data between the computer and external devices (like a keyboard or screen).
- **Addressing Modes (How Data is Found):** This describes how an instruction finds the data it needs to work on. The raw content mentions 'four types of operations' and 'adding contents of R1, R2 & place sum in R3', which hints at different ways data can be specified. Some instructions might work directly with data stored in memory locations, while others might use data stored in the processor's registers. For example, 'Add contents of A to accumulator & store sum back to accumulator' shows an operation where data from a location 'A' is added to a special register (the accumulator), and the result stays in that same register.
- **Instruction Formats (How Instructions are Written):** The way instructions are structured can vary. This is often described by the number of 'addresses' they use to specify where data is located:
- **Zero address instructions:** These instructions don't explicitly mention any data locations. They often work with data implicitly stored in a special structure called a 'push down stack'.
- **One address instruction:** Uses one location to specify data. An example might be 'Add contents of A to accumulator', where 'A' is the data location.
- **Two address instructions:** Uses two locations to specify data. This could involve operations like $B \leftarrow [A] + [B]$, where data from location 'A' is added to data from location 'B', and the result is stored back in location 'B'.
- **Three address instructions:** Uses three locations to specify data. An example is 'Adding contents of R1, R2 & place sum in R3', where data from register R1 and register R2 are added, and the result is placed in register R3.
- **Register Transfer Notation (RTN):** This is a way to describe operations using a shorthand. The 'left hand side of RTN' is the name of a location where a result will be stored, and the 'right hand side of RTN' is what is being calculated or the value itself.